TASK-3 QUICK SORT

Aim: To sort a given set of elements using the quick sort method and determine the time required to sort the elements. The elements can be read from a file or can be generated using the random number generator.

Step-1 : Choose the highest index value has pivot

Step-2 : Take two variables to point left and right of the list excluding pivot.

Step-3 : Left point to the low index

Step-4 : right points to the high

Step-5 : while value at left is less than pivot move right.

Step-6: while value at right is greater than pivot move left

Step-7 : If both step 5&6 doesn't match swap left & right

Step-8 : if left ≥ right, the point where they met is new pivot.

```
# include < stdio.h >
# include < sys/time.>
Void swap (int*x, int*y)
{
 int temp;
  temp = *x ;
  *x = *y;
  *y = temp;
 }
  Void generate_random (int a[], int n)
 {
  int i;
  srand (time(0));
   for (i=0; i<n; i++)
        a[i] = rand() % 1000 ;
       }
```

```
int partition (int a[10], int l, int i, j, P; int h)
{
 i = l ; j = h+1; P = l ;
   do
   {
     do
     {
     i = i+ ;
     } while (a[i] < a[P]); do
      {
          j = j-1 ;
     } while (a[j] > a[P]);
      swap (&a[i], &a[j]);
      } while (i <= j);
      swap (&a[i], &a[j]);
      swap (&a[l], &a[j]);
        return j ;
      }
      int Quicksort (int a[10], int m, int n)
      {
         int s ;
      if (m < n+1)
      {
      s = partition(a, m, n);
      Quick sort (a, m, s-1);
      Quick sort (a, s+1, n);
       return a ;
       }
       }
       int main()
       {
      int a[100000], i, ch, n ;
      Struct time Val t ;
```

```
    double start,end;
    FILE *fp;
  Printf("Enter the type of operation\n");
  Printf("1- Randomly generate numbers & quicksort\n");
printf("2.Enter the number of values to generate and
                                        sort\n");
      scanf("%d",&ch);
       Switch (ch)
       {
case-1:
    fp = fopen("quick sort.txt","w");
   for (i= 10000; i< 100000; i=i+5000)
     {
          generate_random(a,i);
          gettimeofday(&t, NULL);
           Start = t.tv_sec+(t.tv_usec/100000 0●.0);
           Quicksort (a,0,i-1);
           get time of day (&t,NULL);
          end= t.tv_sec +(t.tv_usec/1000000.0);
          Printf("%d\t %.1f\n",i, end- start);
         f print(fp,"%d\t %.1f\n",i,end-start);
       }
         f close (fp);
           break;
 case-2: print f ("Enter the number of values to be
                                    generated\n");
       scanf("%d",&i);
           generate_random(a,i);
            get time of day (&t, NULL);
       Start = t.tv_sec + (t.tv_usec/1000000.0);
```

```
Quicksort (a,0,i1);
get timeofday (&t, NULL);
end: t.tv_sec + (t.tv_usec/1000000.0);
Printf ("%d\t%.1f\n", i, end-start);
Printf (" Sorted numbers are \n");
Printf (" The sorted array is \n");
  for (n=0; n<i; n++)
  Printf ("%d\t", a[n]);
    break;
default:
        Printf ("invalid choice \n")
        }
        return 0;
        }
```

Output: Enter the type of operation

1- Randomly generate numbers and quicksort

2- Enter the no of values to generate & sort

1

10000	0.002030
15000	0.001000
20000	0.001996
25000	0.001996
30000	0.002994
35000	0.002993
40000	0.003990
45000	0.003991
50000	0.004987
55000	0.004986
60000	0.006014
65000	0.006983
70000	0.007980
75000	0.007980
80000	0.007980
85000	0.008977

90000 0.008975
95000 0.008976

~~Process exited after~~

The sorted array is

192 232 242 402 660 718 778 810 813 9

Process exited after 5.296 seconds with return value 0

Press any key to continue.

RESULT : Therefore the above task were sucessfully ve and executed

VEL TECH - CSE	
EX NO.	3
PERFORMANCE (5)	5
RESULT AND ANALYSIS (5)	5
VIVA VOCE (5)	5
RECORD (5)	5
TOTAL (20)	20
SIGN WITH DATE	12/7/25